

Acta fundacional ACB-HUB

1. Integrantes

- Guillermo García León
- Alfonso Masa Calderón
- Manuel Alcaraz Zambrano

2. Team Manager

El Team Manager será responsable de organizar las reuniones del equipo, registrar sus resultados en el diario del equipo y velar por el correcto desarrollo del proyecto.

Se designa a Guillermo García León como Team Manager.

3. Repartición de tareas

En las reuniones, se identificarán las tareas a realizar para satisfacer el siguiente entregable del proyecto.

Cada tarea será evaluada y se le asignará un número en función de su dificultad y del tiempo estimado para completarla.

La escala de peso de las tareas va del 1 al 5. Siendo 1 una tarea sencilla y ligera, y 5 una tarea compleja, que requiere más tiempo y esfuerzo.

Las tareas se asignarán de manera equitativa entre los miembros del equipo.

4. Gestión de cambios en la metodología

Para cambios en la metodología de trabajo del equipo se realizará una reunión. En ella, los miembros del equipo debatirán sobre el cambio, las distintas opciones y ventajas y desventajas de cada una.

La decisión final se tomará mediante votación.

Si se ha decidido realizar el cambio, se modificará este documento y se volverá a firmar.

5. Política de ramas

Se establecen las siguientes ramas:

- **main:** para las entregas, cada merge a main modifica la versión del proyecto, siguiendo la lógica definida en el apartado 5.
- **trunk:** los cambios pasan a esta rama antes de llegar a la rama main.
- **feature:** se creará una rama feature por cada issue o sub issue. Se nombrará de la siguiente forma: feature/task #X, siendo X el identificador de la issue relacionada.
- **bugfix:** se creará una rama bugfix por cada error a solucionar. Se nombrará de la siguiente forma: bugfix/task #Y. Siendo Y el identificador de la issue relacionada con el error a corregir.

Merge entre ramas

Nos ceñiremos a este modelo y no podrá realizar un merge que no esté definido en este apartado.

La fusión entre ramas se hará usando rebase, para mantener un historial limpio.

Se han definido las siguientes posibilidades de merge entre ramas:

- Merge de **trunk** a **main**: se realizará para cada entrega del proyecto.
- Merge de **feature** a **trunk**: cuando la issue relacionada a la rama feature esté completada (que funcione).
- Merge de **bugfix** a **trunk**: cuando la issue de error relacionada a la rama bugfix esté solucionada.

En el caso de las ramas feature y bugfix, una vez realizado el merge, esta se destruirá. En caso de necesidad de cambio a posteriori, se creará una rama nueva.

6. Política de versionado

- Para documentos:

Para cada entrega, si se ha editado el documento, se suma 1 a la versión.

Ejemplo: NombreDeDocumento_v1

- Para el proyecto:

Para el proyecto se considerará la política de versionado de tipo X.Y, donde a X se le sumará 1 por cada Work Item implementado en la rama main y a Y se le sumará 1 por cada error corregido o cambio de una funcionalidad implementada en la rama main.

La primera versión del proyecto es la: v1.0

7. Acuerdo y decisión

Se consideran acuerdos las resoluciones relativas al funcionamiento y metodología del trabajo del equipo, estos quedan grabados en este documento y para su aceptación requiere de la firma de todo el equipo.

Las decisiones son resoluciones menores, como pueden ser aspectos técnicos o el reparto de las tareas. Estos no necesariamente quedan constatados en un documento.

8. Lógica de Issues

Se creará una **issue** por cada Work Item a implementar y por cada tarea a parte como puede ser el cambio del logo de la app, un error a corregir u otro cambio que no necesariamente esté relacionado con un Work Item.

Por otro lado, se usarán **sub issues** para definir cada tarea dentro de un Work Item.

9. Política de commits

Estructura del mensaje de commit

Para redactar los mensajes de commit nos regimos por la estructura de Conventional Commits.

Para definir los tipos de commits nos hemos inspirado en la tipificación usada por Angular¹:

- **build**: Cambios en el sistema de compilación o dependencias externas.
- **ci**: Modificaciones en la configuración de integración continua.
- **cd**: Modificaciones en la configuración de despliegue continuo.
- **docs**: Cambios en la documentación.
- **feat**: Implementación de una nueva funcionalidad.
- **fix**: Corrección de errores.
- **perf**: Mejora de rendimiento.
- **refactor**: Cambios en el código que no añaden ni corrigen funcionalidades.
- **style**: Ajustes de formato o estilo (espaciado, comas, etc).
- **test**: Adición o corrección de pruebas.

En el título de cada commit, se mencionan los cambios que se están realizando. Opcionalmente, contará con una descripción más detallada que explique un poco estos cambios.

¿Cuándo se hace un commit?

Dentro de cada rama de funcionalidad, se realizará un commit por cada sesión de trabajo, aunque el código tenga errores.

¹ <https://github.com/angular/angular/blob/22b96b9/CONTRIBUTING.md#type>

10. Propuesta de CI/CD

Se usará la rama CI/CD para establecer el workflow de Integración y Despliegue continuos, los cambios pasarán a main como un único commit.

11. Política de test

Por cada Work Item implementado se hará al menos pruebas funcionales unitarias y al menos una prueba de interfaz con Selenium.

12. Resolución de conflictos

Conflicto	Resolución
Miembro del equipo no muestra interés en el proyecto y no realiza las tareas adjudicadas ni se presenta a las reuniones.	Preguntar a ese miembro sus intenciones con el proyecto y en caso de no poder contactar con ese miembro o no poder resolver el conflicto, poner en conocimiento al tutor del proyecto.
Se tiene punto de vista distinto en la realización de algún apartado del proyecto	Se debatirán las diferentes posturas y en caso de no llegar a un acuerdo se decidirá por votación.

13. Firmas

Anexo. Histórico de cambios

- 24/11/2025: Eliminado Emilio Espinosa Elías de los integrantes.
- 06/12/2025: Eliminado Alexis Molins López de los integrantes.